The key to fending off these attacks is to recognize that a client (or group of clients) is using more than their fair share of some resource: time or memory or number of connections, and so on. By limiting the resources that any one user is allowed to consume we can reduce the risk of attack. Once a user has authenticated, your application can enforce *quotas* that restrict what they are allowed to do. For example, you might restrict each user to a certain number of API requests per hour, preventing them from flooding the system with too many requests. There are often business reasons to do this for billing purposes, as well as security benefits. Due to the application-specific nature of quotas, we won't cover them further in this book.

> **DEFINITION** A ***quota*** is a limit on the number of resources that an individual user account can consume. For example, you may only allow a user to post five messages per day.

Before a user has logged in you can apply simpler rate-limiting to restrict the number of requests overall, or from a particular IP address or range. To apply rate-limiting, the API (or a load balancer) keeps track of how many requests per second (or minute) it is serving. Once a predefined limit is reached then the system rejects new requests until the rate falls back under the limit. A rate-limiter can either completely close connections when the limit is exceeded, or else slow down the processing of requests; a process known as *throttling*. When a distributed DoS is in progress, malicious requests will be coming from many different machines on different IP addresses. It is therefore important to be able to apply rate-limiting to a whole group of clients